

# Introduction to Requirements Classification

In the context of requirement engineering, the term "classification of requirements" refers to the process of categorizing requirements into different groups or types based on various criteria. Classifications of requirements usually categorize requirements based on their characteristics, nature, or purpose.

The classification of requirements is a dynamic and iterative process, as the needs of stakeholders may evolve over the course of a project.

# User Requirements

Understanding user requirements is crucial in software development as they form the foundation for designing and developing a system that meets the needs and expectations of the end users. user requirements are typically categorized as follows:

1. **Functional Requirements**
2. **Non-Functional Requirements**
3. **User Interface Requirements**
4. **System Requirements**
5. **Operational Requirements**
6. **Regulatory Requirements**

# Functional Requirements

Functional requirements describe the specific functions, features, or behaviors that a system must perform to meet the needs of its users or stakeholders.

## User Interaction

Functional requirements define the core capabilities and features that a system must provide to users. These requirements focus on the specific actions and behaviors that users should be able to perform, such as logging in, searching for information, placing orders, or generating reports. Capturing these functional needs ensures the system can effectively meet the users' primary goals and tasks.

## Data Handling

Functional requirements also encompass the system's ability to properly ingest, store, manipulate, and output data. This includes requirements around data input validation, processing rules, storage formats, retrieval mechanisms, and output formats. Defining clear functional requirements for data handling helps ensure the system can reliably manage the information it needs to support user activities.

## Business Logic

Beyond user interactions and data handling, functional requirements specify the underlying business rules and logic that the system must implement. This can include complex workflows, calculations, decision-making, and policy enforcement. Clearly documenting these functional requirements helps align the system's behavior with the organization's operational needs and ensures the solution can properly automate key business processes.

## System Capabilities

Functional requirements also outline the specific capabilities the system must provide, such as report generation, notification triggers, integration with external systems, and security controls. These requirements help define the full scope of what the system needs to deliver in order to meet the users' and organization's requirements.

# Non-Functional Requirements

Non-functional requirements are just as essential as functional requirements when it comes to building high-quality software systems. These requirements focus on the overall performance, usability, security, and other attributes that the system must possess, rather than specific behaviors or features. Non-functional requirements help ensure that the system meets the needs and expectations of end-users, stakeholders, and regulatory bodies.

1. **Usability:** The system should be easy to use, intuitive, and provide a seamless user experience. This includes aspects like clear navigation, responsive design, and accessible interfaces for users with disabilities.
2. **Performance:** The system should be able to handle the expected workload and user traffic without compromising response times, throughput, or availability. This may involve requirements for server processing power, network bandwidth, and database performance.
3. **Security:** The system must protect sensitive data and user information from unauthorized access, data breaches, and cyber threats. This can include requirements for encryption, access controls, audit logging, and compliance with industry standards and regulations.
4. **Scalability:** The system should be able to accommodate growth in user base, data volume, and transaction rates without significant performance degradation or the need for a complete redesign. This may involve considerations for horizontal scaling, cloud infrastructure, and modular architecture.
5. **Reliability:** The system should be highly available, with minimal downtime and consistent performance, even in the face of hardware failures, network disruptions, or unexpected spikes in usage. This can include requirements for redundancy, failover mechanisms, and automated monitoring and recovery processes.
6. **Maintainability:** The system should be easy to update, debug, and modify, with clear documentation, modular design, and well-defined interfaces. This helps ensure that the system can evolve and adapt to changing business requirements over time.

# Business Requirements

Business requirements are statements of the needs or objectives of a business or organization. They outline what the business aims to achieve through a particular project, initiative, or system.

1

## Identifying Business Goals

The first step in defining business requirements is to clearly identify the overarching goals and objectives of the organization. What is the company aiming to achieve through this new system or software? Whether it's increasing revenue, improving operational efficiency, enhancing the customer experience, or expanding into new markets, these high-level business drivers must be understood and documented.

2

## Aligning with Strategy

Business requirements should be directly aligned with the organization's strategic priorities and long-term vision. The new system or software being developed must support and enable the company's broader business strategy. Careful analysis is needed to ensure the requirements are not just tactical or short-term, but will truly help the organization achieve its strategic goals.

3

## Defining Key Metrics

To measure the success of the new system or software, it's critical to define the key performance indicators (KPIs) and metrics that will be used. These might include revenue targets, cost savings, customer satisfaction scores, user adoption rates, or operational efficiency improvements. Having clearly defined metrics upfront allows the organization to track progress and ensure the business requirements are being met.

# System Requirements

## Functionality

System requirements define the essential functions and capabilities that the system must possess to meet the needs of users and stakeholders. These requirements specify what the system should be able to do, such as processing data, generating reports, integrating with other systems, and providing a user-friendly interface. They ensure that the system is designed to fulfill its intended purposes and deliver the expected value.

## Maintainability

System requirements also encompass the maintainability of the system, which includes factors such as ease of installation, upgradability, troubleshooting, and ongoing support. These requirements ensure that the system can be efficiently maintained, updated, and improved over time, reducing the overall cost of ownership and enhancing the system's longevity.

## Performance

System requirements also address the system's performance characteristics, such as speed, reliability, scalability, and responsiveness. These requirements set benchmarks for the system's operational efficiency, ensuring that it can handle the expected workload and user demands without compromising its functionality or user experience. Performance requirements help to optimize the system's resources and ensure that it meets the desired levels of service.

## Security

Security is a critical aspect of system requirements, as it helps to protect the system, its data, and its users from various threats, such as unauthorized access, data breaches, and cyber attacks. Security requirements define the necessary safeguards, access controls, encryption protocols, and other measures to maintain the confidentiality, integrity, and availability of the system and its information.

# Interface Requirements

Interface requirements in software engineering refer to the specifications and guidelines that dictate how different software components, modules, or systems interact and communicate with each other

1. **User Interface (UI) Requirements:** These requirements describe how users interact with the software system. User interface requirements specify the layout, design, functionality, and usability features of the graphical user interface (GUI) or command-line interface (CLI)
2. **Application Programming Interface (API) Requirements:** APIs define the interfaces and protocols used for communication between different software components or systems. API requirements specify the methods, parameters, data formats, and authentication mechanisms used for invoking and interacting with APIs.
3. **Hardware Interface Requirements:** These requirements define how the software interacts with hardware components such as processors, memory devices, storage devices, input/output (I/O) devices, sensors, and actuators. Hardware interface requirements specify the communication protocols, drivers, and compatibility standards necessary for interfacing with specific hardware platforms
4. **Software Interface Requirements:** Software interfaces describe how software modules, libraries, services, or subsystems interact with each other within the same application or across different applications. Software interface requirements specify the function calls, data structures, parameters, and error handling mechanisms used for inter-module communication and collaboration.
5. **External Interface Requirements:** External interfaces involve interactions between the software system and external entities such as other software systems, databases, web services, networks, protocols, and third-party applications. External interface requirements specify the communication protocols, data exchange formats, security mechanisms, and service-level agreements (SLAs) governing interactions with external entities.
6. **Integration Interface Requirements:** Integration interfaces define the interactions and dependencies between different software components or subsystems during integration and testing. They specify the data flows, synchronization points, error handling mechanisms, and integration testing procedures necessary for integrating and verifying the correct behavior of interconnected software elements.

# Performance Requirements



Performance requirements are critical to ensuring that a system or application can meet the needs of its users and stakeholders. These requirements focus on the system's ability to consistently deliver the expected level of functionality and quality of service, even as demands on the system grow and evolve over time.

Reliability is a core performance requirement, ensuring that the system operates flawlessly and without interruption. Scalability is also vital, enabling the system to handle increasing numbers of users, transactions, or data volumes without degradation in performance. And responsiveness - the system's ability to provide quick, smooth, and seamless interactions - is essential for delivering an optimal user experience.

By carefully defining and measuring performance requirements, organizations can build systems that are robust, efficient, and able to maintain high levels of service delivery even as needs change. This allows them to meet and exceed user expectations, stay competitive, and provide maximum value.



# Regulatory Requirements

Regulatory requirements are rules, laws, or guidelines set by governments, industry organizations, or other authorities that dictate what a software system must comply with in terms of safety, security, privacy, and other important aspects



## Legal Compliance

Adhering to all relevant laws and regulations governing the software industry.



## Security Standards

Meeting the highest security standards to protect user data and privacy.



## Compliance Balance

Striking a balance between compliance requirements and user experience.